SHELF & SOUL · RAMALLAH

Shelf & Soul

Implementation and File Report

Project files, delivered functionality, validation results, and prototype boundaries.

This report inventories the application files created for the Shelf & Soul library-management prototype and describes the current implementation against the supplied Software Requirements Specification.

Prepared 19 October 2026	Project Next.js 15 · React 19 · TypeScript
Specification Shelf_and_Soul_SRS_with_Technologies.docx, v3.0	Prototype storage Browser Local Storage

Project at a glance

12 project files inventoried	4 previewable user roles	0 dependency audit findings

Shelf & Soul is a responsive web prototype for the library and bookshop. It uses Next.js and TypeScript, with a browser Local Storage data layer, sample books and members, and role-aware screens for a Manager, Receptionist, Warehouseman, and Customer.

The interface includes role-specific dashboards, a searchable catalog and book details, circulation, reservation queues, inventory and stock history, member summaries, notifications, and default policy settings.

How to run: In the project folder, run **npm install**, then **npm run dev**. Open the local URL printed by Next.js. Use the top-right selector to preview each workspace.

Files worked on

Complete inventory of application source, configuration, and project documentation files created or updated for this implementation:

File	Purpose and work delivered
app/page.tsx	Main interactive application: role navigation and dashboards, catalog search/filter/sort, book details, circulation and borrow-request screens, reservations, inventory and member tables, notifications, and settings.
app/globals.css	Shared visual tokens and responsive styling: palette, reusable buttons, badges, cards, tables, mobile drawer, and focus states.
app/layout.tsx	Next.js root layout, stylesheet loading, page title, and metadata.
lib/library-service.ts	Local Storage and circulation service: policy validation, borrow requests and approvals, returns and fines, reservation queues and holds, notices, and stock movements.
lib/demo-data.ts	Seed books, members, loans, requests, reservations, stock movements, and notices.
lib/types.ts	Shared TypeScript types for roles, books, members, loans, requests, reservations, stock movements, notices, and application state.
package.json	Project scripts and Next.js, React, TypeScript, and type dependencies; PostCSS override resolves dependency advisories.
package-lock.json	Reproducible npm dependency lockfile.
tsconfig.json	Strict TypeScript compiler settings, Next.js plugin, and import alias.
next.config.ts	Next.js configuration entry point.
next-env.d.ts	Next.js TypeScript environment declarations.
README.md	Local startup instructions, prototype scope, persistence details, and production limitations.

The supplied SRS document was read as the implementation source; it was not modified. Build output, installed packages, and TypeScript cache are generated artifacts rather than hand-maintained project files.

Implemented requirements and behavior

Area	Current implementation	SRS references
Role workspaces	Role selector previews Manager, Receptionist, Warehouseman, and Customer; navigation and dashboard content change by role.	FR-DSH-01–05; UIR-08–10
Catalog	Search title, author, category, and ISBN; filter by category and availability; sort by title, author, or price; view details and availability.	FR-CAT-01–03; UIR-12–15, 19
Borrowing	Customer requests available books; Receptionist approves or rejects pending requests. Approval records issue and a 14-day due date.	FR-LOAN-01–05
Returns and fines	Receptionist records returns. Calendar days overdue determine fines at ₪1 per day, capped at book price; member balance updates.	BR-01–04, 10; FR-LOAN-06; FR-FIN-01
Reservations	Eligible Customer and Receptionist can queue reservations. Queue head is offered available copy for a three-calendar-day hold; expired holds advance when data loads or another reservation is made.	BR-05–08; FR-RES-01–07
Inventory	Inventory summary, low-stock indicators, available and loaned quantities, and stock movement history.	BR-18; FR-INV-01, 06, 07, 09
Members	Staff views show member contact, membership status/expiry, active loans, and fine balance.	FR-MEM-04–06
Notifications	Recent notices on dashboard and in notification popover; notices can be marked read.	FR-NOT-01, 03
Policies	Settings view displays default circulation, fine, reservation, and low-stock values.	FR-SET-01–02 (default display)
Responsive UI	Desktop sidebar, mobile slide-in drawer, responsive cards, and horizontally scrollable tables.	UIR-08, 16, 20, 33

Validation performed

- TypeScript: npm run typecheck — passed.
- Production build: npm run build — passed; home page is statically prerendered.
- Dependencies: npm audit — zero vulnerabilities after the PostCSS override.
- Browser walkthrough: role switching, catalog navigation, borrow approval, and return flow exercised.
- Business rule: returning a seeded loan six calendar days overdue added ₪18 to the member balance.
- Responsive check: catalog opened at 390px viewport width; no horizontal page overflow detected.

Browser walkthroughs are manual checks; no automated unit-test suite is currently configured.

Scope still outstanding

This is a working prototype slice, not a complete delivery of every requirement in the SRS. Areas remaining for future implementation include:

• Registration, login, secure authentication, password reset, profile editing, and session timeout.

• Manager book/category creation and editing, soft-delete/restore workflows, and image-upload validation.

• Renewals, reservation cancellation and collection actions, direct receptionist issue, and return-condition capture.

• Fine administration, payments and partial payments, simulated online checkout, sales receipts, and order cancellation.

• Supplier management, purchase suggestions, manager PO approval, and receiving.

• Staff management, service-enforced permissions, audit log, report exports, and editable settings.

• Public About, Contact, Help/FAQ, and full production-ready accessibility verification.

Production boundary
Local Storage is device-local and can be edited or cleared. The prototype role selector is not authentication or access control. Do not use this prototype for real customer records or payments. Production still requires a trusted server API, PostgreSQL, secure authentication and server-side authorization, payment-provider integration, audit enforcement, and automated backups.